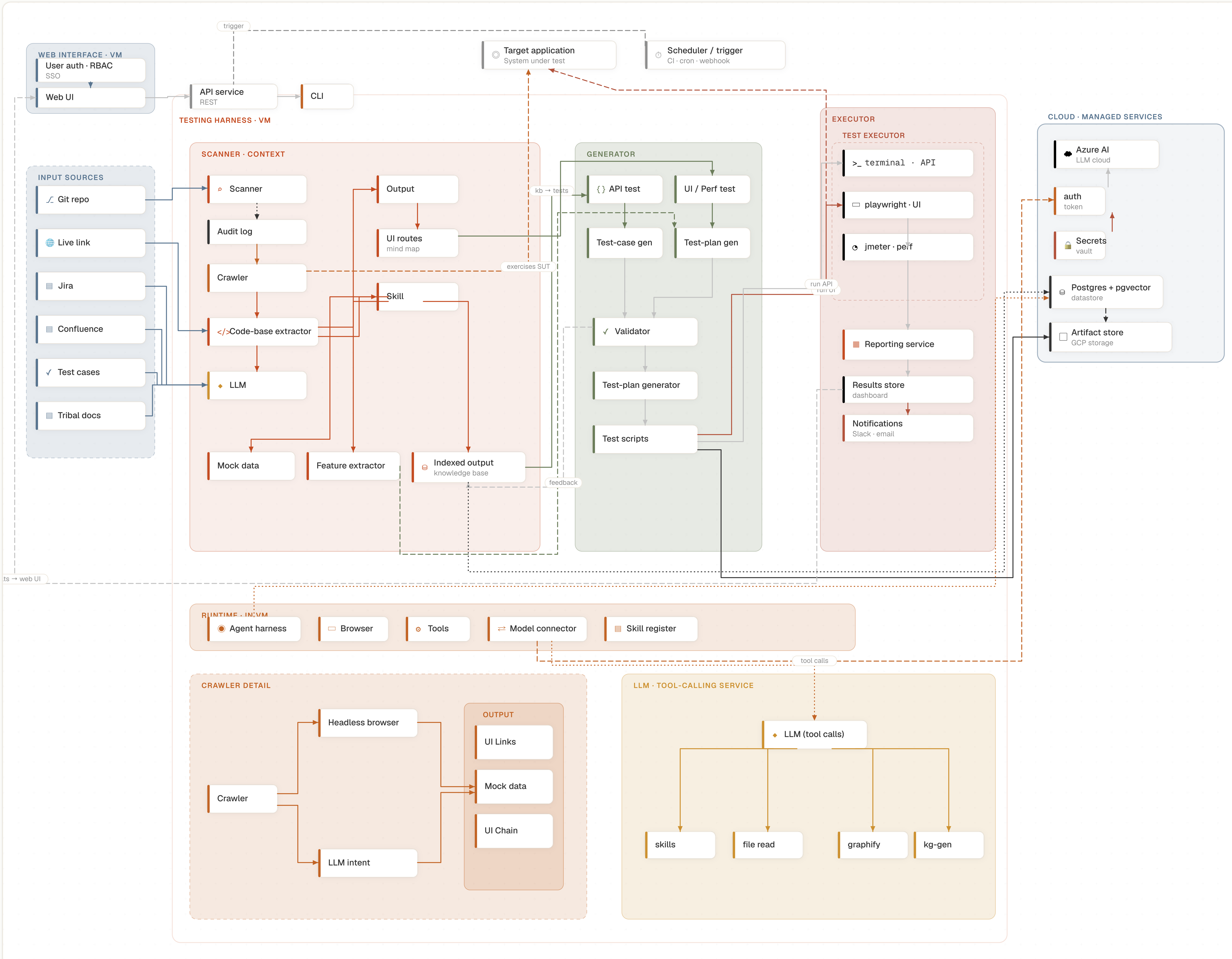
SYSTEM ARCHITECTURE · V11

AI Testing Harness

One VM container runs the testing harness — scanner, generator, executor and runtime. Cloud (Azure) holds the managed services it depends on: Azure AI (LLM), auth, secrets, the Postgres + pgvector datastore and the artifact store. Input sources and the target application stay external.



Ops / hardening

Config store	per-app settings: base URLs · exemptions · creds map
LLM cache	rate-limit / quota (Azure)
Backup / DR	Postgres + artifact store
Ingress	load balancer (web UI)

Tech stack · agent calling · deployment · datastore

Packages / tech

`crawler`	Playwright + Crawlee
`code extractor`	graphify (OpenHarness) + AST
`LLM + tools`	kg-gen (DSPy + LiteLLM)
`model connector`	HTTP API (OpenAI-compat)
`indexer`	Node (.mjs)
`API tests`	requests + curl + pydantic
`UI tests`	Playwright
`perf tests`	JMeter
`openapi tests`	Node + js-yaml
`doc sources`	kg-gen + pypdf + requests

Agent calling

`harness`	OpenHarness (openharness-ai)
`tool_service`	graphify · get_skill · file read · kg-gen
`LLM advisor`	adviseOn() → model-api-connector
`connector`	getClient('provider').chat() → model API
`LiteLLM`	openai / gemini / kimi / ollama (optional)

Deployment / VM

`1 VM`	CLI + agent harness + Playwright browser + tools + connector
`web UI`	separate VM (interface)
`inputs`	EXTERNAL to the VM
`future`	Docker image · multi-app output isolation
`offline`	local mirrors · Ollama · vendored libs

Datastore

`database`	PostgreSQL + pgvector — KB, results & vector recall
`object store`	GCP Cloud Storage — test scripts & artifacts (S3-compatible)

Input sources · Scanner · context · Generator · Executor · Runtime · in VM · LLM tool-calling · Datastore / infra

data flow · feedback / external · shared infra